

Library Management System

15-Day End-to-End Development Plan

Django REST Framework + React + TypeScript + PostgreSQL

Generated: October 2026

Days 1-4

Backend Foundation & Bug Fixes

Days 5-10

Frontend Build (all pages)

Days 11-13

Polish, Testing & Security

Days 14-15

Docs, Deploy & Handoff

High Overview

Day	Title	Focus Area
1	Project Setup & Bug Fixes	Backend Foundation
2	Fix Borrowing Core + Book Return Flow	Backend - Borrowing App
3	Books App Cleanup + Pagination + Logout	Backend - Books & Users Apps
4	Backend Polish + API Documentation	Backend - Final cleanup
5	Frontend Setup + Routing + Auth Pages	Frontend - Foundation
6	Book Listing + Book Detail Pages	Frontend - Books
7	User Dashboard - My Requests + History	Frontend - User Features
8	Admin Dashboard - Borrow Requests Management	Frontend - Admin Features
9	Admin - Books & Authors CRUD	Frontend - Admin Features
10	Admin - Users Management + Dashboard Home	Frontend - Admin Features
11	UI Polish - Styling, Toasts, Loading States	Frontend - UX & Polish
12	Integration Testing + Bug Fixes	Full Stack - Testing
13	Performance + Security Hardening	Backend & Frontend - Production Readiness
14	Documentation + Deployment Preparation	Project - Docs & Deploy
15	Final Review, Polish & Handoff	Project - Completion

Tips for Beginners

1. Work in order.

Each day builds on the previous. Don't skip ahead to the frontend while the backend is broken.

2. Test after every task.

Don't batch 3 hours of coding and then test. Test each small piece as you go. It's faster overall.

3. Use Postman for the backend.

Every time you add or fix a backend endpoint, test it in Postman before touching the frontend.

4. Commit daily.

One meaningful commit at the end of each day. Small commits inside the day are fine too. Never end a day with uncommitted working code.

5. Don't Google first - read the error.

Read the full stack trace or error message before searching. It usually tells you exactly what is wrong.

6. Django migrations are a chain.

Never delete migration files. If you make a mistake, create a new migration to fix it, don't edit old ones.

7. React re-renders are not bugs.

If data doesn't refresh after an action, you likely forgot to refetch or update state. Add a refetch call after successful mutations.

8. Keep components small.

If a component file gets longer than 150 lines, split it into smaller pieces. Easier to debug, easier to reuse.

9. The 'available_quantity' is the source of truth.

Don't add a separate is_available boolean to the DB. Derive it in the serializer: `available_quantity > 0`. One source of truth = fewer sync bugs.

10. Understand before you copy.

When you look up how to do something, read the explanation, don't just copy the code. You will have to debug it later.

Day 1

Project Setup & Bug Fixes

Backend Foundation

09:00 - 09:30

- Read through the entire codebase + review.md so you know every problem before touching anything.

09:30 - 10:00

- Fix DEBUG env bug in settings.py -> `os.getenv('DEBUG','False').lower()=='true'`

10:00 - 10:45

- Fix password hashing bug -> change `create()` to `create_user()` in `RegisterSerializer`.

10:45 - 11:15

- Fix login empty-check -> change 'and' to 'or' in `LoginAPIView`.

11:15 - 12:00

- Remove `print(username, password)` from `LoginAPIView`. Fix MAILERS -> EMAIL_BACKEND.

12:00 - 13:00 - LUNCH BREAK

13:00 - 13:30

- Run the server, test register + login with Postman/Thunder Client to confirm fixes work.

13:30 - 14:30

- Add requirements.txt -> run 'pip freeze > requirements.txt' inside the venv.

14:30 - 15:30

- Install django-cors-headers, add to INSTALLED_APPS + MIDDLEWARE, set CORS_ALLOWED_ORIGINS for localhost:5173.

15:30 - 16:00

- Git commit: 'fix: critical security and config bugs'

16:00 - 17:00

- Study: read Django Token Auth docs + DRF APIView docs (bookmark, don't memorise).

Day 2

Fix Borrowing Core + Book Return Flow

Backend - Borrowing App

09:00 - 09:30

- Re-read the borrowing/views.py patch() method carefully. Understand the TOCTOU bug by tracing execution line by line.

09:30 - 10:30

- Fix race condition: move availability check inside transaction.atomic(), use select_for_update() to lock book row.

10:30 - 11:00

- Test the fix: open two Postman tabs, fire both approval requests simultaneously for same book, confirm quantity decrements correctly.

11:00 - 12:00

- Add return_book endpoint: PATCH /api/v1/borrowing/<id>/return/ -> sets returned_at, increments available_quantity with select_for_update.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Add returned_at = DateTimeField(null=True, blank=True) to BorrowRequest model. Create and run migration.

14:00 - 14:30

- Fix duplicate URL name in borrowing/urls.py. Add the new return URL. Fix history URL to point to BorrowingHistoryAPIView.

14:30 - 15:30

- Fix BorrowingHistoryAPIView: remove broken .filter() line. Implement proper history list (filter by user for non-staff, all for admin).

15:30 - 16:00

- Fix history permission classes: implement has_permission() method in both AdminOnlyBorrowingHistory and UserOnlyBorrowingHistory.

16:00 - 16:30

- Test all borrowing endpoints end-to-end in Postman: create request -> approve -> return -> check quantity restored.

16:30 - 17:00

- Git commit: 'fix: borrowing availability sync, return flow, permissions'

Day 3

Books App Cleanup + Pagination + Logout

Backend - Books & Users Apps

09:00 - 09:30

- Add logout endpoint to users/views.py: DELETE request that calls `request.user.auth_token.delete()` and returns 200.

09:30 - 10:00

- Add logout URL to users/urls.py. Test: login -> get token -> call logout -> call a protected endpoint -> confirm 401.

10:00 - 11:00

- Add pagination: create a custom `PageNumberPagination` class (`page_size=10`). Apply to `BooksAPIView.get()` and `AdminOnlyBorrowingAPIView.get()`.

11:00 - 11:30

- Rename `Books.user` FK to `added_by` -> create migration, update serializer and admin.

11:30 - 12:00

- Add `rejected_by` and `rejection_reason` fields to `BorrowRequest` model. Add admin PATCH endpoint for reject action.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Implement reject logic in `AdminOnlyBorrowingAPIView`: accept action field ('approve'/'reject') in PATCH, route accordingly.

14:00 - 15:00

- Add Genre CRUD endpoints to `books/views.py` and `books/urls.py` (they exist in the model but have no view).

15:00 - 15:30

- Test all Books endpoints (CRUD) + Genre endpoints in Postman.

15:30 - 16:00

- Ensure `BooksSerializer` returns `is_available` computed field (`available_quantity > 0`) as a read-only `SerializerMethodField`.

16:00 - 17:00

- Git commit: 'feat: logout, pagination, reject flow, genre endpoints, is_available field'

Day 4

Backend Polish + API Documentation

Backend - Final cleanup

09:00 - 10:00

- Add drf-spectacular (pip install drf-spectacular) for auto API docs. Add to INSTALLED_APPS, configure in settings.py, add /api/schema/ and /api/docs/ URLs.

10:00 - 11:00

- Write basic tests for the register endpoint and login endpoint in users/tests.py using Django TestCase.

11:00 - 12:00

- Write tests for borrow approve + return flow in borrowing/tests.py. Test that available_quantity decrements and restores correctly.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Run all tests: python manage.py test. Fix anything that fails.

14:00 - 15:00

- Add proper error messages and consistent response structure across all views: {success, message, data} pattern.

15:00 - 16:00

- Review all endpoints in Postman one final time. Document the base URLs in a simple API_REFERENCE.md in the backend folder.

16:00 - 17:00

- Git commit: 'docs/test: api docs, basic test suite, consistent responses'. Backend is now feature-complete.

Day 5

Frontend Setup + Routing + Auth Pages

Frontend - Foundation

09:00 - 09:30

- Install frontend dependencies: `npm install react-router-dom axios zustand`. This gives you routing, HTTP client, and state management.

09:30 - 10:00

- Plan the page structure on paper: Login, Register, Home (book list), Book Detail, My Requests, Admin Dashboard, Admin Books, Admin Users.

10:00 - 11:00

- Set up React Router in `main.tsx`. Create placeholder page components for all 8 pages above. Each is just an empty div with the page name for now.

11:00 - 12:00

- Build the Login page UI: form with username + password fields, submit button. Style it cleanly with plain CSS (no new library needed).

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Wire Login page to the API: POST to `/auth/login/`, store the token in Zustand store and `localStorage` on success, redirect to Home.

14:00 - 15:00

- Build Register page UI and wire it to POST `/auth/register/`. On success, auto-login (store token) and redirect.

15:00 - 16:00

- Create a `PrivateRoute` component that checks if token exists in Zustand store. Wrap protected routes with it. Redirect to `/login` if not authenticated.

16:00 - 16:30

- Add a Navbar component: shows username + logout button when logged in. Logout calls DELETE `/auth/logout/` and clears the store.

16:30 - 17:00

- Git commit: 'feat: frontend setup, auth pages, routing, private routes'

Day 6

Book Listing + Book Detail Pages

Frontend - Books

09:00 - 10:00

- Create an `api.ts` file in `src/`: set up an axios instance with `baseUrl` and an interceptor that attaches the token from `localStorage` to every request.

10:00 - 11:30

- Build the Home page (Book List): fetch `GET /api/v1/books/` with axios, display books in a responsive grid. Show title, author, genre, `available_quantity`, and `is_available` badge.

11:30 - 12:00

- Add a search/filter input on the Home page that filters the displayed book list client-side by title or author name.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:30

- Build the Book Detail page: fetch `GET /api/v1/books/<id>/`, display full book info. Add a 'Request to Borrow' button that is disabled if `available_quantity` is 0.

14:30 - 15:30

- Wire the 'Request to Borrow' button: show a small form asking for `requested_days` (1-30), `POST` to `/api/v1/borrowing/user/` on submit, show success/error toast.

15:30 - 16:00

- Add pagination controls to the Home page (previous/next buttons using the paginated API response).

16:00 - 17:00

- Git commit: 'feat: book listing, book detail, borrow request from UI'

Day 7

User Dashboard - My Requests + History

Frontend - User Features

09:00 - 10:30

- Build 'My Requests' page: fetch GET /api/v1/borrowing/user/, display borrow requests in a table with status badges (Pending = yellow, Approved = green, Rejected = red, Cancelled = grey).

10:30 - 11:30

- Add a 'Return Book' button on approved rows. Wire it to PATCH /api/v1/borrowing/<id>/return/. Disable it if returned_at is already set.

11:30 - 12:00

- Show due date: calculate approved_at + approved_days and display it next to each approved request. Highlight overdue rows in red.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Build 'My Profile' page: fetch GET /auth/me/ (add this endpoint if missing), display user info. Keep it simple - no edit for now.

14:00 - 15:30

- Build 'Borrowing History' page: fetch GET /api/v1/borrowing/history/, show completed (returned) borrows in a read-only table.

15:30 - 16:30

- Add loading spinners and empty-state messages to all three pages (spinner while fetching, '- No requests yet -' when list is empty).

16:30 - 17:00

- Git commit: 'feat: user dashboard, my requests, return book, borrowing history'

Day 8

Admin Dashboard - Borrow Requests Management

Frontend - Admin Features

09:00 - 09:30

- Create an AdminRoute component: same as PrivateRoute but also checks is_staff from the Zustand store. Redirect non-admins to Home.

09:30 - 11:00

- Build Admin Borrow Requests page: fetch GET /api/v1/borrowing/, display all requests in a table with user, book, requested_days, status columns.

11:00 - 12:00

- Add Approve action: clicking 'Approve' opens a small modal asking for approved_days input, then fires PATCH /api/v1/borrowing/<id>/ with {action:'approve', approved_days:N}.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Add Reject action: clicking 'Reject' opens a modal with a rejection_reason text field, fires PATCH with {action:'reject', rejection_reason:'...'}

14:00 - 15:00

- Add status filter tabs at the top: All / Pending / Approved / Rejected. Filter the table client-side by selected tab.

15:00 - 16:00

- Add a stats bar at the top of the admin page: Total Requests, Pending, Approved, Rejected - calculated from the fetched data.

16:00 - 17:00

- Git commit: 'feat: admin borrow request management with approve/reject'

Day 9

Admin - Books & Authors CRUD

Frontend - Admin Features

09:00 - 10:30

- Build Admin Books page: table listing all books with columns for title, author, genre, quantity, available_quantity, actions (Edit / Delete).

10:30 - 11:30

- Add Book: clicking 'Add Book' opens a modal form with all fields (title, description, isbn, author, genre, quantity). POST to `/api/v1/books/` on submit.

11:30 - 12:00

- Edit Book: clicking 'Edit' pre-fills the same modal form with existing data. PUT to `/api/v1/books/<id>/` on submit.

12:00 - 13:00 - LUNCH BREAK

13:00 - 13:45

- Delete Book: clicking 'Delete' shows a confirmation dialog. DELETE to `/api/v1/books/<id>/` on confirm.

13:45 - 15:00

- Build Authors page: table of all authors with Add / Edit / Delete. Wire to `/api/v1/books/authors/` endpoints.

15:00 - 16:00

- Build Genres page (simple): list of genres with Add / Delete. Wire to `/api/v1/books/genres/` endpoints.

16:00 - 17:00

- Git commit: 'feat: admin books and authors CRUD UI'

Day 10

Admin - Users Management + Dashboard Home

Frontend - Admin Features

09:00 - 10:00

- Build Admin Users page: fetch GET /auth/users/, display table of all users with username, email, phone, is_staff, is_active columns.

10:00 - 11:00

- Build Admin Dashboard Home: a summary overview page with 4 stat cards - Total Books, Total Users, Pending Requests, Active Borrows. Fetch data from relevant endpoints.

11:00 - 12:00

- Add a simple bar/pie chart using a lightweight library (recharts: npm install recharts) showing borrow requests by status.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Build the admin sidebar/nav: links to Dashboard, Books, Authors, Genres, Borrow Requests, Users. Highlight the active link.

14:00 - 15:00

- Make the admin layout responsive: sidebar collapses to a hamburger menu on mobile.

15:00 - 16:00

- Test the full admin flow end-to-end in the browser: login as admin -> view dashboard -> add book -> approve a borrow request.

16:00 - 17:00

- Git commit: 'feat: admin users page, dashboard overview, admin nav'

Day 11

UI Polish - Styling, Toasts, Loading States

Frontend - UX & Polish

09:00 - 10:00

- Install react-hot-toast (npm install react-hot-toast). Replace all alert() and console.log with toast.success() / toast.error() across every page.

10:00 - 11:30

- Do a full visual audit of every page. Ensure consistent font sizes, spacing, button styles, and colour usage across the app.

11:30 - 12:00

- Add a consistent page header component (page title + breadcrumb) reused across all pages.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Handle all error states: if an API call returns 400/401/403/404/500, show a meaningful toast or inline message instead of silently failing.

14:00 - 15:00

- Make the book grid and tables responsive. Test on a 375px (mobile) viewport in browser DevTools.

15:00 - 16:00

- Add skeleton loading placeholders for the book list and borrow requests table so the UI doesn't feel empty while loading.

16:00 - 17:00

- Git commit: 'style: toasts, loading states, responsive layout, visual consistency'

Day 12

Integration Testing + Bug Fixes

Full Stack - Testing

09:00 - 10:30

- Full user journey test: Register -> Browse books -> Request borrow -> Wait for admin approval -> Return book -> Check history. Do this entirely through the UI.

10:30 - 12:00

- Full admin journey test: Login as admin -> View pending requests -> Approve one -> Reject one -> Add new book -> Edit existing book. Do this through the UI.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Test edge cases: try borrowing an unavailable book (should be blocked), try accessing admin routes as a regular user (should redirect), try registering with duplicate username.

14:00 - 15:00

- Fix every bug and UX issue you found during testing above. Don't move on until all the journeys work cleanly.

15:00 - 16:00

- Run python manage.py test on the backend. Fix any failing tests.

16:00 - 17:00

- Git commit: 'fix: integration testing bug fixes'

Day 13

Performance + Security Hardening

Backend & Frontend - Production Readiness

09:00 - 10:00

- Add `select_related` / `prefetch_related` to all queriesets that touch related objects (BorrowRequest -> book + user, Books -> author + genre). This reduces N+1 queries.

10:00 - 11:00

- Add database indexes on frequently filtered fields: BorrowRequest.status, BorrowRequest.user, Books.title. Add via Meta class on models + migrate.

11:00 - 12:00

- Review all endpoints for missing authentication. Use Postman to hit every endpoint without a token and confirm it returns 401.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Add rate limiting to the login endpoint to prevent brute force (django-ratelimit: pip install django-ratelimit). Limit to 10 attempts per minute per IP.

14:00 - 15:00

- On the frontend: store the token only in memory (Zustand) plus httpOnly cookie approach, OR at minimum ensure the app clears tokens on logout properly.

15:00 - 16:00

- Add `ALLOWED_HOSTS` and ensure `SECRET_KEY` is a real secret in `.env`. Review `.gitignore` - confirm `.env` is excluded.

16:00 - 17:00

- Git commit: 'perf/security: N+1 fixes, indexes, auth hardening, rate limiting'

Day 14

Documentation + Deployment Preparation

Project - Docs & Deploy

09:00 - 10:30

- Write a proper README.md in the project root: project description, features list, tech stack, setup instructions (clone -> install -> .env -> migrate -> run), API overview.

10:30 - 11:30

- Write backend/.env.example with all required keys listed (values blank). Commit this file so others know what to set.

11:30 - 12:00

- Build the frontend production bundle: npm run build. Fix any TypeScript errors that surface during build.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Configure Django to serve the frontend build as static files (or document that they're served separately). Add gunicorn to requirements.txt.

14:00 - 15:00

- Set up a free-tier Render.com or Railway.app deployment for the backend. Set all environment variables in the dashboard.

15:00 - 16:00

- Deploy the frontend to Vercel or Netlify (free). Point VITE_API_URL env variable to the deployed backend URL.

16:00 - 17:00

- Smoke test the deployed app: register, login, browse books. Git tag: git tag v1.0.0. Push tags.

Day 15

Final Review, Polish & Handoff

Project - Completion

09:00 - 10:00

- Re-read review.md from Day 0. Verify every single critical and high-severity issue is now fixed.

10:00 - 11:00

- Do one final full end-to-end walkthrough on the deployed app (not localhost). Test on mobile via your phone.

11:00 - 12:00

- Check the Django admin panel at /admin/ - ensure BorrowRequest, Books, Authors, Users are all registered and display cleanly.

12:00 - 13:00 - LUNCH BREAK

13:00 - 14:00

- Clean up dead code: remove the commented-out BorrowingHistory model, remove unused imports across all files, delete the generate_plan.py script from root.

14:00 - 15:00

- Write a CHANGELOG.md listing what was built, what was fixed, and what could be improved further (JWT tokens, email notifications, fine / overdue tracking).

15:00 - 16:00

- Final git commit: 'chore: cleanup, changelog, final review'. Push everything to main. Share the deployed URLs.

16:00 - 17:00

- Rest. You shipped a full-stack library management system in 15 days. Review what you learned and write 5 things you would do differently next time.